Software Requirements

And Design Specification

for



Sprint 3

Prepared by Maicol Parr, Bailey Hall, Dakota Middleton, Tate Hendrix, Kekoa New

University of West Florida

12/03/2023

Demo Video: https://youtu.be/TfljHqGGFmE

Copyright © 2023 by apinto. Permission is granted to use, modify, and distribute this document.

Table of Contents

1. Introduction **1**
1.1 Description 1
1.2 Team Contributions **1**
1.3 Document Conventions **2**
2. Overview of Project Characteristics **3**
2.1 User Groups and Characteristics **3**
2.2 Operating Environment **3**
2.3 Assumptions and Dependencies **3**
3. User Interfaces **4**
4. Class Design **6**
4.1 Class Diagram 6
4.2 Responsibility of Each Class **7**
5. System Features **9**
5.1 Home Screen and Navigation **9**
5.2 POS and Its Transactions **10**
5.3. Inventory Display and Management 13
6. Data Storage and Schema **17**
6.1 Data Sources **17**
6.2 Formats and Schema **17**
7. Known Issues and Bugs **19**
Appendix A: GitHub Information **19**
Appendix B: Required Reports **21**
Appendix C: Glossary **21**

1. Introduction

1.1 Description

The purpose of this application is to serve as a Point of Sales system for a small to a mid-sized retail store. The system will have two main components, the Point of Sale side which focuses on completing transactions, and the inventory side which allows employees to manage the store's inventory. In the Inventory system, an employee can add, edit, remove, and search for products from the store's internal product record for use in customer transactions. Furthermore, users also can reactivate previously deleted products within the Inventory system and view statistics on previous transactions. In the Point of Sales system, an employee can add and remove items from the current transaction, search for products, apply and manage product discounts, clear transactions, and complete a customer's transaction.

1.2 Team Contributions

Maicol Parr has assisted in the management of the Trello Board, the Required Software Requirements and Design Document, Project Planning, Documentation, the UML Class Diagrams, setting up the branches and issues on GitHub, and the development of the system front-end GUI and backend core class components. Maicol actively communicated with his fellow team members to integrate the inventory and transactional management systems with the GUI to ensure functionality through user testing, error handling, and user input management for the front-end components. Maicol has contributed to the development of the "Add to Inventory", "Edit Inventory", "GUI Navigation", and the "Add to Transaction List" features of the application. He has also assisted with the creation and testing of the "Remove from Inventory", "Complete Checkout", "Apply Discounts", and "Database Integration" features of the application as well. Finally, Maicol worked to develop the database connection error notification system, the remove or edit product by row selection feature, the product removal confirmation window, the display discount feature, and the company branding feature.

Bailey Hall has assisted in the maintenance of the Trello Board, the Required Software Requirements and Design document, updating the UML class diagrams, using GitHub features, the feature to remove items from the inventory, the design for the program's GUI using Balsamiq, and taking meeting notes when necessary. Furthermore, Bailey contributed to the development of the "Remove from Inventory" functionality within the application.

Dakota Middleton contributed to the creation of the Trello Board and assisted in the creation of the RSR, and RGC. He also assisted in the rough draft of the design template for the GUI/Design layout for the project. Upon completion of these tasks, he began to work on drafts for the GUI, including the home screen and POS screen. Coding contributions also included adding to the test file for the product class. SPRINT 2: Worked with the team reviewing what we would work on during the second sprint. Worked on the discount functionality, creating linking methods and the development of the "pop-up GUI" which will allow the user to apply discounts to items in the current transaction.

Tate Hendrix has assisted in the production of the group assets such as the group calendar, time log form, the Google Drive group folder, and group discord. He contributed to the development of the project by developing the Inventory class methods, assisting in the development of the Transaction class methods, and writing the Product class and all its methods. Tate set up the Inventory and Product class tester files and helped develop tests for each of them. Furthermore, Tate contributed to the “Search Inventory” and “Database Integration” features of the application. During the final sprint, Tate expanded the database to include two new tables with a relational connection to store transaction data and the item data for those transactions. He then created the stats function to display the general and item statistics in a popup window when prompted.

Kekoa New has contributed to the creation of the initial UML design, the point of sales panel, as well as the initial GUI. Koa assisted with the creation and organization of the Trello board. Contributions to the point of sales panel include: styling the JFrame components, creating functionality to add items to the current transaction list, and integrating the transaction list with the inventory components. Koa also researched the necessary Java and JFrame topics required to implement the design of the program. Koa actively communicated with his fellow team members to affirm the design and implementation of the program. During sprint two, Koa contributed to the void transaction and checkout features. This includes logging receipts of transactions to a file and reducing the quantity of items from the inventory when a purchase is confirmed. Koa also attended group meetings to communicate with his teammates and assisted with other feature branches when necessary. During sprint 3, Koa refactored the transaction, POS GUI, checkout window, and discount classes to use product copy constructors. Koa attempted to implement an observer pattern in the refactor, but was unable to. Koa also helped debug the program at various points during the sprint.

1.3 Document Conventions

No document conventions for this document are currently required. Future conventions will be included here if required in future iterations.

2. Overview of Project Characteristics

2.1 User Groups and Characteristics

The main user group using this product is the employees working the register at a small to mid-sized retail store. A store employee can scan the products that a customer would like to buy and have the correct transaction information show up on the screen so that the products can be sold successfully. The employee will also be able to manage the store's internal inventory when new products arrive at the store and are bought by the customer.

2.2 Operating Environment

The software must be able to operate on any Macintosh, Windows, or Linux operating system that has the latest version of Java JDK installed and can run native Java applications. It can be run on any computer with the ability to run Java applications and has the technical capabilities to run and compile the code. For now, this application requires an IDE or bash command to compile and run the application itself, with instructions on how to do so included on this project's GitHub repository The link to download the latest version of Java JDK can be found here. The user must also add the database JAR file contained within the GitHub repository to their classpath within their preferred development environment to establish a connection to the database. The link to learn more about the SQLite database management system can be found here. Finally, this application assumes that it will only be used by employees, and will not be used by anyone else including customers.

2.3 Assumptions and Dependencies

This application is intended for a small to mid-sized retail company for handling transactions and inventory management. It is currently intended that all employees have access to both the transaction and inventory management systems, as no user role for managers or specific positions currently exists. It is intended that the employees are entrusted with the ability to use both functions and are appropriately trained on the software before use.

The system depends on a cash payment method during transactions. It also assumes that all items in the inventory are given a unique SKU and that every item can be rung up in the POS system. Furthermore, this application is dependent on a SQLite database instance and the "SQLite-jdbc-3.39.2.0.jar" file needed to establish a connection to the database within the application. The SQLite database is included within the repository and is assumed to run alongside the application. It is also assumed that the user has appropriately configured their development environment to include the JAR instructional file in their classpath. Furthermore, it assumes that the database for the application is not edited by the employee in any way. All changes to the database must be made by the application itself, and should not be edited by a user or employee. Finally, it is assumed that the application itself will be used on a computer where the newest version of Java's JDK is installed and can compile and run native Java applications.

3. User Interfaces

The user will first be presented with a Home screen with two buttons that will either lead them to the POS screen or the Inventory screen, which they would choose depending on how they are about to use the system. On the Inventory screen, they should be able to see all items currently in the store's inventory and they should be able to search, add, edit, remove, and reactivate items. They should also be able to see statistics for all transactions or particular products within the last thirty days. On the POS screen, they should be able to scan/enter a unique SKU that is then added to a current transaction that is being carried out.

When items are added to the transaction, the user should be able to remove them or void the entire transaction. After the items are added, the user will be able to apply a discount by entering a product's SKU in the current transaction and applying the percentage or value amount that they would like to take off. When items are finished being added, the user should be able to see the total and finish the transaction. After the transaction is completed, the user will be able to append the transaction to a text log containing all transactions for later use. After, they can either stay on the POS screen or go back to the Home Menu to continue using the application.

The GUI interface provides error message responses to the user when appropriately needed. Within the Add to Inventory Screen, error messages appear in the form of bold red text labels towards the bottom of the window to indicate an error. While on the POS Screen, errors appear in the form of pop-up error labels indicating the corresponding error message. These errors include but are not limited to:

Inventory Management System

- Add to Inventory Screen & Edit Inventory Screen
    - Leaving any input text boxes blank
    - Adding a product with a quantity of less than zero
    - Adding a product with a negative cost or price, or a price or cost that exceeds a limit
    - Adding a duplicate product that already exists within the active or inactive inventory lists
    - Not including only decimal values for the cost, price, and quantity values
- Reactivate Product Window
    - Attempting to open the window when there are no products available to reactivate
    - Attempting to reactivate a product not within the reactivation list
    - Attempting to reactivate a product that does not exist

Point of Sales Screen

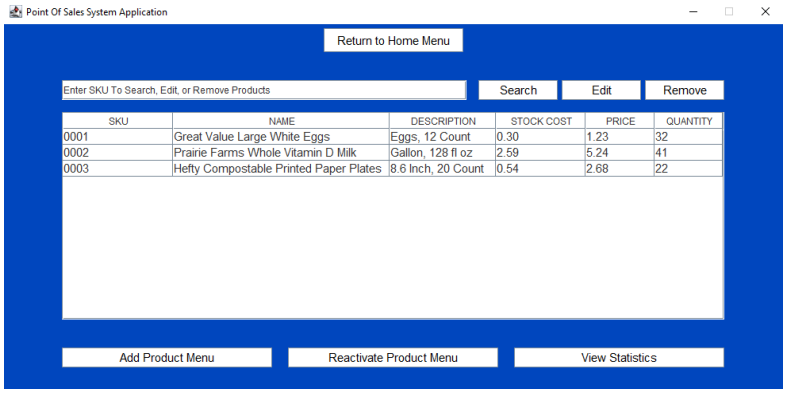
- Adding/Removing from Transaction
    - Attempting to add a product that does not exist within the inventory
    - Attempting to remove a product that does not exist within the Transaction List
    - Attempting to add more items to the transaction list than within the Inventory
    - Attempting to remove more items from the transaction list than within the Transaction List
- Checkout Customer during Transaction
    - Attempting to checkout with an empty cart
- Discount Screen
    - Not selecting a product in the transaction list to discount
    - Attempting to input a discount value that is less than 1 or greater than the product's current price
    - Attempting to input a discount percentage that is less than 1% or greater than 100%
    - Leaving any input text boxes blank or any option buttons unselected

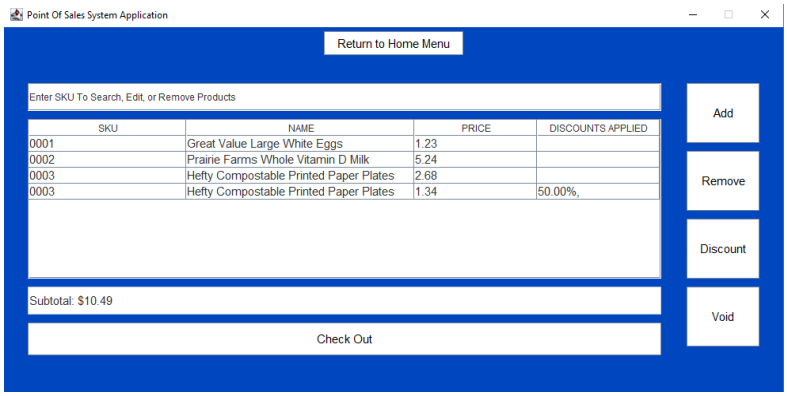
- Attempting to input a discount value that is not a decimal value




MID
BUY
Welcome
Continue to:
Inventory
POS
Back to Screen Selection
JMenuBar
Search
Edit
Remove
JTable
When you click search, the table only shows items that match the search
Or:
Add New Item
Back to Screen Selection
SKU#123456
Item Name
Price
iPhone 17
900
iPhone charger
100
JTable or JList?
Total: $1,000.00
Check Out
Discount
Remove
Void
MID
BUY

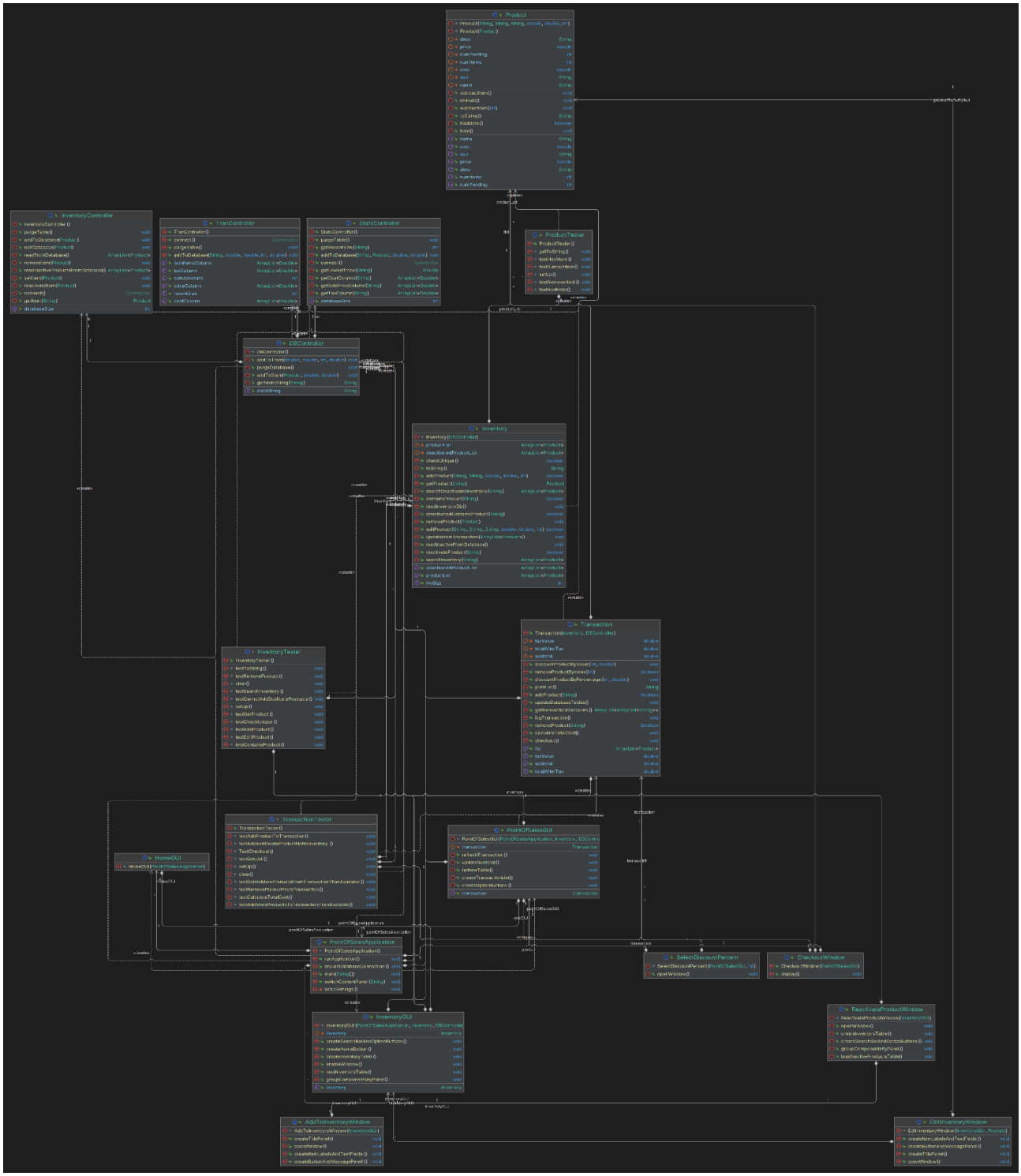
Point Of Sales System Application
MID
BUY
Welcome
Continue to:
Inventory
POS


Point Of Sales System Application
Return to Home Menu
Enter SKU To Search, Edit, or Remove Products
Search
Edit
Remove
SKU
NAME
DESCRIPTION
STOCK COST
PRICE
QUANTITY
0001
Great Value Large White Eggs
Eggs, 12 Count
0.30
1.23
32
0002
Prairie Farms Whole Vitamin D Milk
Gallon, 128 fl oz
2.59
5.24
41
0003
Hefty Compostable Printed Paper Plates
8.6 Inch, 20 Count
0.54
2.68
22
Add Product Menu
Reactivate Product Menu
View Statistics


Point Of Sales System Application
Return to Home Menu
Enter SKU To Search, Edit, or Remove Products
SKU
NAME
PRICE
DISCOUNTS APPLIED
0001
Great Value Large White Eggs
1.23
0002
Prairie Farms Whole Vitamin D Milk
5.24
0003
Hefty Compostable Printed Paper Plates
2.68
0003
Hefty Compostable Printed Paper Plates
1.34
50.00%,
Subtotal: $10.49
Check Out
Add
Remove
Discount
Void

4. Class Design

4.1 Class Diagram



4.2 Responsibility of Each Class

Name of Class	Class Description
Product	Creates and maintains Product objects.
HomeGUI	Creates the elements of the Home screen. Allows for navigation to the Inventory and the POS Screens.
InventoryGUI	Creates the elements of the Inventory screen. Handles visual components of the inventory, such as updating the JTable containing the items in the inventory.
PointOfSalesGUI	Creates the elements of the POS screen. Handles visual components of the transaction, such as updating the JTable containing the items in the transaction.
Transaction	Handles transaction functionality such as adding and removing items from the transaction list and calculating the total price of the transaction.
AddToInventoryWindow	Creates a window for users to add a new product to the Inventory. Handles error checking for user inputs and allows for new products to be added internally.
Inventory	Holds and manages products in the inventory. Allows for the creation, deletion, and management of inventory products. Relies on the DBController class to manage the Inventory database.
PointOfSalesApplication	Implements all GUI classes. Acts as the main frame for the application. Initializes the Point of Sales Application. When run, the program will appear on the screen. Acts as a main driver for the application.
EditToInventoryWindow	Creates a window for users to edit products within the Inventory. The edited product attributes are loaded into this window for the user to edit and confirm. Handles error checking for user inputs and allows for current products to be edited internally.
CheckoutWindow	Creates a window for the user to view a summary of their transaction, the sales tax, and the total plus tax. Confirming the purchase decrements the quantity from the inventory, writes a receipt to a log file, and presents a confirmation dialog to the user informing them that their purchase was successful.
SelectDiscountPercent	Prompts a menu to the user that allows them to enter the item they want to discount, and a percent value they would like to remove from the item. The transaction list is then updated with the new value, which updates the subtotal value to the new price.

DBController	Forms a connection to the Inventory.db external SQLite Database and handles all communication with the InventoryController, StatsController, and TransController database controller objects.
InventoryController	Handles all database operations for the Inventory Table. Handles the addition, deletion, editing, deactivation, reactivation, and deletion within the Inventory.
TransController	Handles the storage for all transactional data within the transaction table of the database. Maintains the transaction list and the list of all products in each transaction.
StatsController	Handles the retrieval of statistics for all previous transactions and each product within the inventory. Returns the statistics to be displayed in the GUI.
ReactivateProductWindow	Creates a window for users to search for and reactivate previously deleted products within the inventory. Displays a message if opened by the user with no products to reactivate.

5. System Features

5.1 Home Screen and Navigation

5.1.1 Description

The home screen will serve as the welcome screen and allow users to select what they would like to do. Users should also be able to switch between the POS and inventory screens when necessary.

5.1.2 Related User Groups

Since the program is designed for employees, all employees of a company will have access to the home screen.

5.1.3 Functional Characteristics

- ➔ Users can select Inventory and the Inventory screen will appear - Complete
- ➔ Users can select POS and the POS screen will appear - Complete
- ➔ Users can navigate between all screens - Complete

5.1.4 Non-Functional Characteristics

- ➔ Welcomes the user to the application - Complete
- ➔ Branding looks similar to Best Buy - Complete

5.1.5 Testing

User tests:

Open the program. Click on the Inventory button on the left. Click on the home button at the top of the screen to go back. Click on the POS button on the right. Click on the home button at the top of the screen to go back.

Automated tests:

Test	What does it do?
N/A	N/A

0% of tests were run
0% of tests succeeded
0% of tests failed

5.2 POS and Its Transactions

5.2.1 Description

The POS will be able to handle customer transactions. Employees should be able to “scan” or type the SKU for a certain product and it will be added to a transaction from which the

customer will checkout. An employee will also be able to remove items from a transaction, void/clear a transaction, apply a discount to a transaction, and complete customer transactions. Furthermore, an employee will also be able to search for items within the inventory list. Finally, employees will be able find a file containing receipts of all transactions made since the software was installed.

5.2.2 Related User Groups

Since the program is designed for employees, all employees will have access to the POS screen. This is under the assumption that only employees use this software and that it faces no direct customer interaction.

5.2.3 Functional Characteristics

- ➔ Users can add items to the current transaction list - Complete
- ➔ If the same item is added twice, that is reflected on the screen - Complete
- ➔ Users can view the transaction total - Complete
- ➔ Users can checkout customers - Complete
- ➔ Users can view a log of all previous and current transactions - Complete
- ➔ Users can remove items from the current transaction - Complete
- ➔ Users can void the entire transaction - Complete
- ➔ Users can add discounts to particular items - Complete
- ➔ Users can view all discounts applied to particular items - Complete

5.2.4 Non-Functional Characteristics

- ➔ Should be able to calculate the total very quickly to finish up customer transactions efficiently - Complete
- ➔ Should be able to see all options (remove, void, etc. buttons) on the screen very clearly - Complete
- ➔ Branding looks similar to Best Buy - Complete

5.2.5 Testing

User tests:

Before starting, ensure that at least one product is within the inventory with a quantity of 1.

Steps:

1. Click on the POS button from the home screen. In the “SKU” box at the top left of the screen, type “0001” and click add.
2. Type “0001” and click add again, an error message stating that there are not enough products within the inventory should be displayed.
3. Type “0001” into the bar again and attempt to remove the product twice. It will be successful the first time but should display an error message the

second time stating that there are not enough products within the transaction list to be removed.

4. Type “0002” into the bar and click add, an error message should appear stating that the product cannot be found.
5. Type “0001” into the search bar and add the item back into the transaction.
6. Click the discount button, and type the SKU of the item in the text field at the top of the popup window. Type the percentage to discount the item in the lower text field.
7. Click apply discount. The price of the item will be changed based on the inputted discount.
8. Click the void button to clear the entire transaction.
9. Repeat step 5 and click the checkout button. This will display a summary of the current transaction.
10. Click the cancel purchase button. This will dispose of the checkout window.
11. Click the checkout button again and click the confirm purchase button. This will display a dialog informing you that the purchase was completed.
12. Navigate to the Inventory window to confirm the quantity of the items in the transaction was properly decremented in the inventory.

Automated tests for Transaction Class:

Test	What does it do?
setUp()	Creates a new inventory object, adds products to the inventory, and creates a new transaction object.
clear()	Clears the inventory and resets the inventory database DBController for other tests.
testAddProductToTransaction()	Ensures that existing products within the inventory can be added to the transaction.

testRemoveProductFromTransaction()	Ensures that products within the transaction list can be removed from the list
testAddAndDeleteProductNotInInventory()	Ensures that Products that do not exist within the inventory cannot be added to the transaction and that products that are not within the transaction list cannot be removed from the transaction
testAddMoreProductsToTransactionThanAvailable()	Ensures that products can only be added to the transaction if there is enough quantity of that product to do so
testDeleteMoreProductsFromTransactionThanAvailable()	Ensures that products can only be deleted from the transaction if there is enough quantity of that product to do so. Cannot delete a product that does not exist within the transaction.
testCalculateTotalCost()	Ensures that the total cost of the transaction is the sum of the prices of all products within the transaction list
testGetList()	Ensures that the Transaction List adequately stores all products added to the transaction.
TestCheckout()	Ensures the quantity

	of the products in the inventory are properly updated after a transaction is checked out.

100% of tests were run
100% of tests succeeded
0% of tests failed

5.3. Inventory Display and Management

5.3.1 Description

The inventory screen will first display all of the items currently in the store's inventory along with the options to search for and add, remove, or edit specific items. This allows the store to add new products to the inventory as they arrive, and manage their in-store collection of items.

5.3.2 Related User Groups

Since the program is designed for employees, all employees will have access to the Inventory screen. This is under the assumption that only employees use this software and that it faces no direct customer interaction.

5.3.3 Functional Characteristics

- ➔ Users can add new items to the Inventory list - Complete
- ➔ Users can view all items within the inventory list - Complete
- ➔ Users can remove items from the Inventory list - Complete
- ➔ Users can edit items in the Inventory list - Complete
- ➔ Users can search the Inventory - Complete
- ➔ Users can store all product and inventory information within a SQLite Database - Complete
- ➔ Users can reactivate previously deleted products - Completed
- ➔ Users can view statistics on all transactions and specific products within the last 30 days - Complete

5.3.4 Non-Functional Characteristics

- ➔ All products should be displayed clearly and neatly in a chart/grid/table that displays at minimum the product's name, SKU, and price - Complete
- ➔ When the user wants to add to the Inventory, it will pop up as a form in a separate window to make inputting the product's information easy - Complete
- ➔ Branding looks similar to Best Buy - Complete

5.3.5 Testing:

User tests:

1. Click on the Inventory button from the Home screen.
2. Select the Add to Inventory button.
3. In the form, type “Product1”, “Description”, “500.00”, “1000.00”, and “1” in the respective text fields.
4. Click submit. The new product should appear within the inventory table list within the inventory screen.
5. In the form, attempt to add a new product but test the error handling methods. Leave some text boxes blank, attempt to add non-decimal values for the cost, price, and quantity, attempt to add a new product with a quantity below one, and attempt to add a duplicate product. All of these should prevent a new product from being added and should display an error message.
6. Type the SKU of the item from Step 3 into the search bar and click the edit button. This will open the “Edit Inventory Window.”
7. Change the value in each of the product fields.
8. Click the edit item button. Ensure the specified product has the updated values.
9. Repeat step 6 and attempt to enter invalid values into the text fields to test error handling.
10. Type “Product1” into the text bar and press search. The resulting table should display the product with SKU 0001’s information. Press search again to clear the search criteria
11. Type “0001” into the search bar and press delete. Once the product is deleted, attempt to add a new product. This product should have the SKU number “0002”
12. Open the Reactivate Product window, the product “0001” should be shown in the window. Now, type 0001 into the search bar and press the reactivate button. Once you click confirm and close the window, the reactivated product will appear in the inventory window for use in future transactions.
13. Navigate to the POS screen and complete a transaction with any of the items previously entered in the inventory. Once you have checked out, navigate back to the inventory window.
14. Click the “statistics” button. There should be a summary of the store’s profit margins as well as other financial information regarding transactions made within the last 30 days.

Automated tests for Inventory Class:

Test	What does it do?
setUp()	Creates new inventory objects before each test case
clear()	Clears the inventory database after each test case to ensure that the tests are working with an empty inventory
testAddProduct()	Ensures that a new product can be added to the internal inventory record
testCannotAddDuplicateProducts()	Ensures that duplicate products cannot be added to the inventory
testCheckUnique()	Ensures that each item within the inventory has a unique SKU number. Products cannot take the SKU numbers of previously deleted products
testRemoveProduct()	Ensures that products can be removed from the inventory list
testEditProduct()	Ensures that products' attributes can be edited within the inventory list
testSearchInventory()	Ensures that products can be searched for within the inventory
testContainsProduct()	Ensures that the inventory can be queried to determine whether it contains a product or not
testGetProduct()	Ensures that the inventory can return a product based on a provided SKU number
testToString()	Ensures that the toString method of the inventory class functions appropriately.

100% of tests were run
100% of tests succeeded

0% of tests failed

Automated tests for Product Class:

Test	What does it do?
setUp()	Creates a new Product object before each test case
testSubtractItem()	Ensures that product's quantity values can be deducted by a provided amount. Also ensures that the product's holds are removed when the quantity is deducted in cases of user checkout.
testAddHold()	Ensures that holds can be added to existing products for use in user transactions.
testRemoveHold()	Ensures that product holds can be removed from existing products for checkout purposes.
testHasMore()	Ensures that no more products can be bought or put on hold than are available within the inventory.
getToString()	Ensures that the Product's data is successfully stored within the class object

100% of tests were run
100% of tests succeeded
0% of tests failed

6. Data Storage and Schema

6.1 Data Sources

An external SQLite database named "Inventory.db" is used for this application. This database uses the SQLite database format system and requires the included JAR file to be added to the user's IDE's classpath. Furthermore, the "res" folder on GitHub contains a MidBuy image logo for use in

the README.md file, the Inventory.db database, and the “sqlite-jdbc-3.39.2.0.jar” file needed to establish a connection to the database within the application.

6.2 Formats and Schema

The Inventory.db file follows the following format:

- Each row in the database’s Inventory Table contains the product’s SKU number, name, description, product cost, product price, quantity, and activity status.

- The inventory database is read at the beginning of each program launch to be imported to an internal memory state within the application. The database is updated when the user adds, edits, or removes from the inventory. Updates to the database are also made when a transaction is completed, and each item’s quantity is reduced by the number of products bought. This is to ensure redundancy when saving products to validate that all updates to the inventory system are included.

- SKU numbers are provided to new products in ascending sequential order. Products deleted within the Inventory are marked as inactive but remain in the database for later reference and possible reactivation. Any new products will be assigned a new SKU number based on the total number of products contained within the database, including the deactivated ones. This allows for all SKU numbers to be unique, and ensures that the system maintains consistency regarding the inventory management technique.

- The Transactions table maintains a list of all individual transactions and contains a transaction ID, total cost, total price, number of items, date of transaction, and sales tax amount. The Product_Stats table maintains a list of individual products within each transaction, keeping track of the transaction ID, product SKU, name, cost, price, sold price, and tax amount. This table follows a one-to-many integration with the transaction table for use in statistical analysis within the Inventory Management System.

Database Schema:

- Inventory Table:

```
Create table inventory (
   sku      TEXT,
   name     TEXT,
   desc     TEXT              not null,
   cost     DOUBLE(18, 2),
   price    DOUBLE(18, 2),
   numItems INTEGER,
   active   BOOLEAN default TRUE not null
);
```

- Product_Stats Table:
    -
```
Create table product_stats (
   transID   TEXT,
   sku       TEXT,
   name      TEXT,
   cost      double,
   price     double,
   soldPrice double,
   date      TEXT
```

```
);
```

- Transactions Table:
    - Create table transactions (
      transID TEXT,
      totalCost double,
      totalPrice double,
      numItems integer,
      date TEXT
      );

7. Known Issues and Bugs

- Non-Intended Issues
    - The UI currently does not scale correctly for certain screen resolutions. The UI has been scaled down from full-screen to temporarily fix this issue. - Low Priority

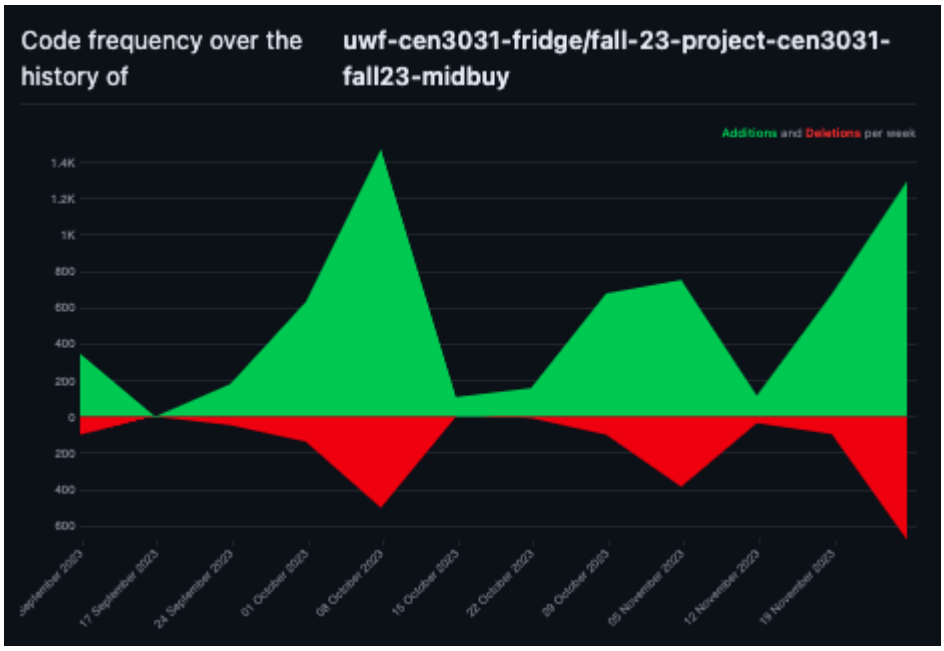
Appendix A: GitHub Information

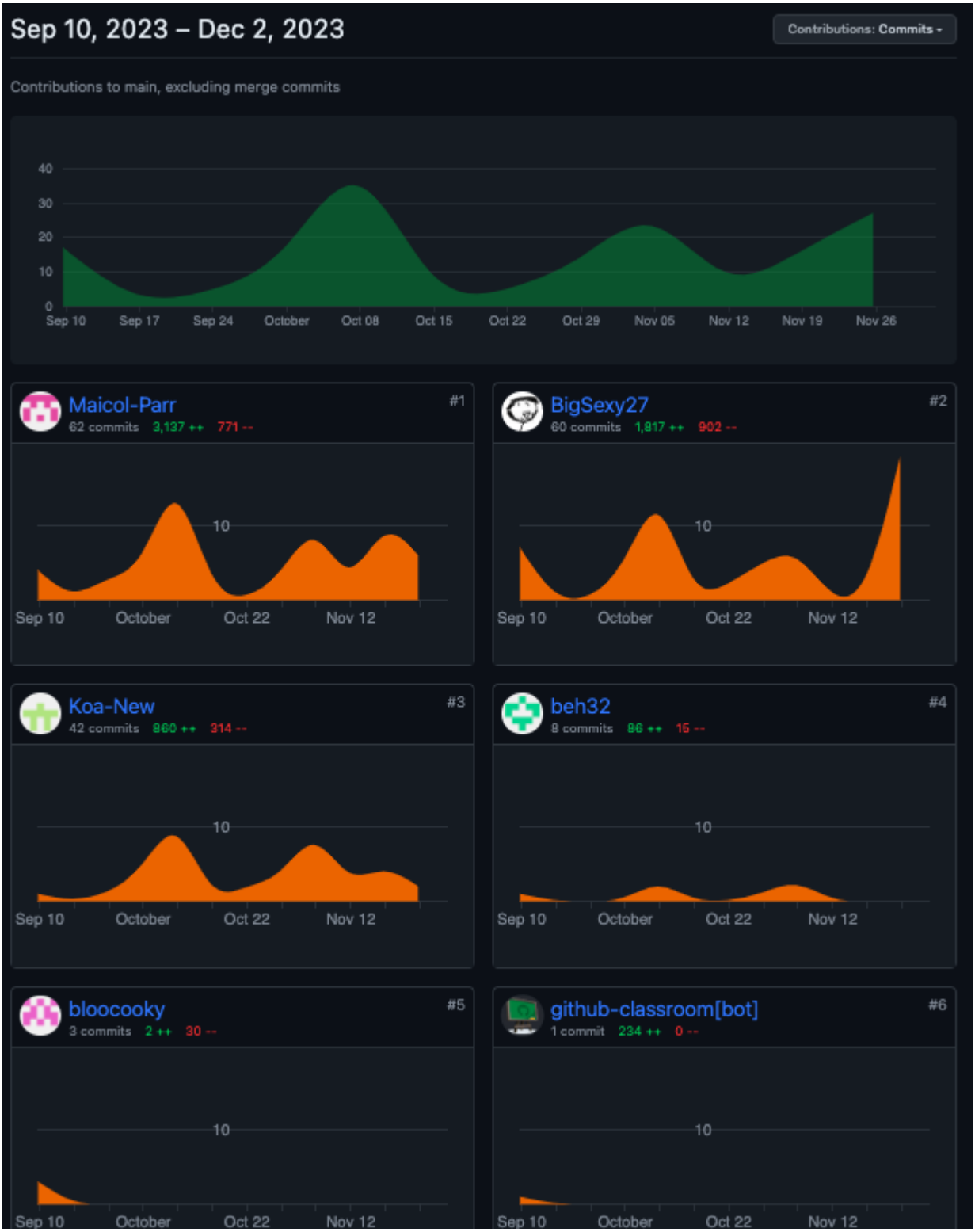
Repository Name: fall-23-project-cen3031-fall23-midbuy

Repository Link: https://github.com/uwf-cen3031-fridge/fall-23-project-cen3031-fall23-midbuy

Features Used:

- Creating and Merging Branches
- Handling Pull Requests with confirmation requests from at least one other team member
- Issues for each User Story in Trello
    - Issues include the user story name and the condition of satisfaction
    - Create a new branch for each issue opened
    - Allows feature to be labeled and merged into main when completed
- Protection of the Main branch, requiring at least one other team member to approve the pull request before merging into main.
- Division of the GitHub repository into subdivided features following a common GitHub repository configuration
- Including tags on each issue, denoting where in the process the feature currently resides (i.e. enhancement, bug, duplicate, etc.)




Sep 10, 2023 – Dec 2, 2023
Contributions: Commits
Contributions to main, excluding merge commits
40
30
20
10
0
Sep 10
Sep 17
Sep 24
October
Oct 08
Oct 15
Oct 22
Oct 29
Nov 05
Nov 12
Nov 19
Nov 26
Maicol-Parr
#1
62 commits 3,137 ++ 771 --
BigSexy27
#2
60 commits 1,817 ++ 902 --
Koa-New
#3
42 commits 860 ++ 314 --
beh32
#4
8 commits 86 ++ 15 --
bloocooky
#5
3 commits 2 ++ 30 --
github-classroom[bot]
#6
1 commit 234 ++ 0 --
Sep 10
October
Oct 22
Nov 12

Appendix B: Required Reports

No reports are currently generated by or through this program. Future reports will be included here if required in forthcoming iterations.

Appendix C: Glossary

- GUI: Graphical user interface
- POS: Point of Sales
- JDK: Java Development Kit, used for developing applications in Java
- SKU: A unique identifier for a product within the inventory
- UML: Unified Modeling Language, used for modeling class diagrams
- IDE: Integrated Development Environment, a programming environment for software development
- JFrame: A part of the Java Swing Library, used for generating interactive windows for Java-based applications
- JPanel: A part of the Java Swing Library, used to hold and organize components for Java-based applications
- JAR: A Java file configuration that is used to package multiple classes together in one container for ease of distribution and use by other developers.
- SQLite: A database management system that is run and maintained by a local machine. The database is stored in a ".db" file and can be accessed using the SQLite3 command line interface.